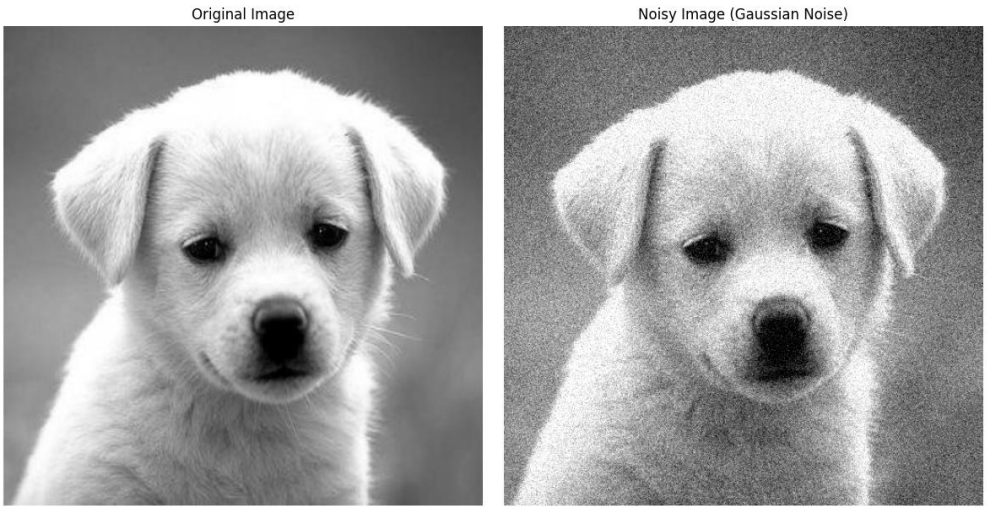
Name of Experiment: Frequency Domain Filtering Using Butterworth and Gaussian Low Pass Filters

Introduction: In this experiment, a **512×512** grayscale image was used and Gaussian noise was added to it. The noisy image was then transformed into the frequency domain using the Fourier Transform. Two low pass filters, namely the 4th order Butterworth Low Pass Filter and the Gaussian Low Pass Filter, were applied to reduce the noise and smooth the image. Finally, the filtered images were displayed and compared to analyze the performance of both filters.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from skimage.transform import resize
from skimage.util import random_noise
from skimage.io import imread
import cv2

image_original = imread('/content/5226523_orig.jpg', as_gray=True)
image_original = resize(image_original, (512, 512),
anti_aliasing=True)

image_original = image_original / image_original.max()

image_noisy = random_noise(image_original, mode='gaussian', var=0.01)

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
ax = axes.ravel()

ax[0].imshow(image_original, cmap='gray')
ax[0].set_title('Original Image')
ax[0].axis('off')

ax[1].imshow(image_noisy, cmap='gray')
ax[1].set_title('Noisy Image (Gaussian Noise)')
ax[1].axis('off')

plt.tight_layout()
plt.show()
```

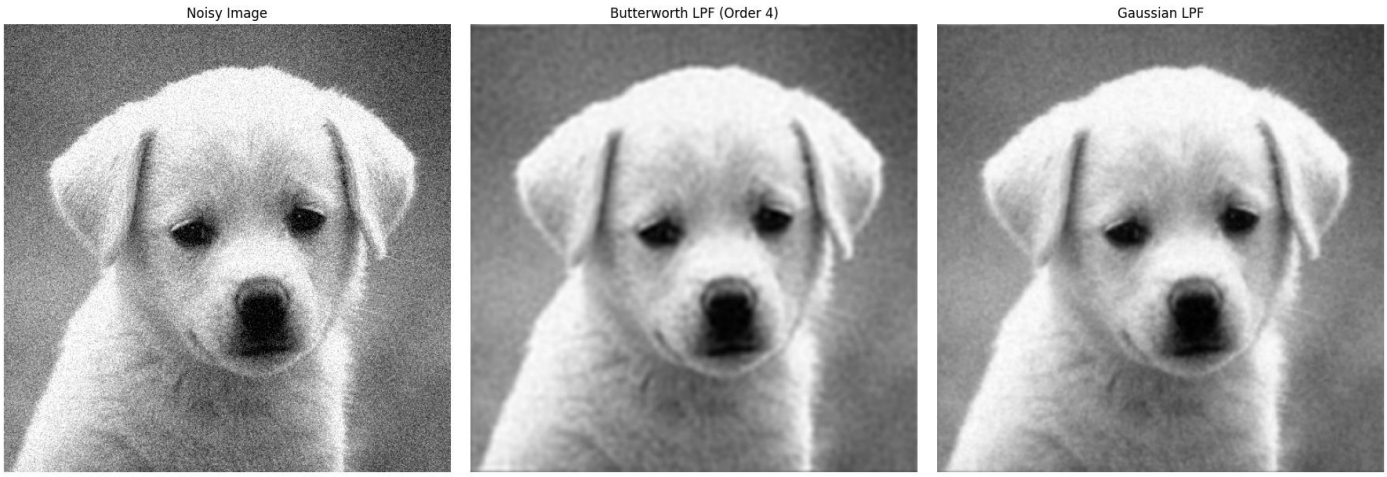
Output Image-1:



Task: Apply 4th order Butterworth

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

f_transform = np.fft.fft2(image_noisy)
f_transform_shifted = np.fft.fftshift(f_transform)

def butterworth_lowpass_filter(shape, cutoff, n):
    P, Q = shape
    U, V = np.meshgrid(np.arange(Q) - Q / 2, np.arange(P) - P / 2)
    D = np.sqrt(U**2 + V**2)
    return 1 / (1 + (D / cutoff)**(2 * n))

def gaussian_lowpass_filter(shape, cutoff):
    P, Q = shape
    U, V = np.meshgrid(np.arange(Q) - Q / 2, np.arange(P) - P / 2)
    D = np.sqrt(U**2 + V**2)
    return np.exp(-(D**2) / (2 * (cutoff**2)))

D0 = 50
order = 4

butter_lpf = butterworth_lowpass_filter(image_noisy.shape, D0, order)
gauss_lpf = gaussian_lowpass_filter(image_noisy.shape, D0)

f_butter = f_transform_shifted * butter_lpf
f_gauss = f_transform_shifted * gauss_lpf

img_butter = np.abs(np.fft.ifft2(np.fft.ifftshift(f_butter)))
img_gauss = np.abs(np.fft.ifft2(np.fft.ifftshift(f_gauss)))

img_butter = img_butter / (img_butter.max() + 1e-12)
img_gauss = img_gauss / (img_gauss.max() + 1e-12)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(image_noisy, cmap='gray')
axes[0].set_title('Noisy Image')
axes[0].axis('off')

axes[1].imshow(img_butter, cmap='gray')
axes[1].set_title(f'Butterworth LPF (Order {order})')
axes[1].axis('off')

axes[2].imshow(img_gauss, cmap='gray')
axes[2].set_title('Gaussian LPF')
axes[2].axis('off')

plt.tight_layout()
plt.show()

cv2.imwrite('output_1_butterworth.png', (img_butter *
255).astype(np.uint8))
cv2.imwrite('output_2_gaussian.png', (img_gauss * 255).astype(np.uint8))
```

Output Image-2:



Discussion: The noisy image contains random Gaussian noise that reduces image quality. After applying the Butterworth Low Pass Filter, the image becomes smoother while preserving some edge details. The Gaussian Low Pass Filter also removes noise effectively and produces a smoother and more natural-looking image. Comparing both outputs shows that low pass filters help reduce high-frequency noise in the frequency domain, although some image details may become slightly blurred after filtering.